Miscellaneous

23O. Miscellaneous Patterns (5 problems)

Standalone algorithms that don't fit neatly into one pattern category but are frequently asked.

23.176 Kadane's Algorithm (Maximum Subarray Sum)

JAVASCRIPT Copy

```javascript
function maxSubArray(nums) {
  let maxSum = nums[0], currentSum = nums[0];
  for (let i = 1; i < nums.length; i++) {
    currentSum = Math.max(nums[i], currentSum + nums[i]);
    maxSum = Math.max(maxSum, currentSum);
  }
  return maxSum;
}
```

Time: O(n). **Space:** O(1).

23.177 Boyer-Moore Voting (Majority Element)

JAVASCRIPT Copy

```
  for (let i = 1; i < nums.length; i++) {
    if (count === 0) { candidate = nums[i]; count = 1; }
    else if (nums[i] === candidate) count++;
    else count--;
  }
  return candidate;
}
```

23.178 Reservoir Sampling (Random selection from stream)

Select k items from a stream of unknown length with equal probability:

JAVASCRIPT Copy

```
function reservoirSample(stream, k) {
  const reservoir = stream.slice(0, k);
  for (let i = k; i < stream.length; i++) {
    const j = Math.floor(Math.random() * (i + 1));
    if (j < k) reservoir[j] = stream[i];
  }
  return reservoir;
}
```

23.179 Product of Array Except Self

Build left and right products. No division needed.

JAVASCRIPT Copy

```
function productExceptSelf(nums) {
  const n = nums.length;
  const result = Array(n).fill(1);
```

```
  for (let i = n - 1; i ≥ 0; i--) { result[i] *= right; right *= nums[i];
  return result;
}
```

Time: O(n). **Space:** O(1) excluding output.

23.180 Top K Frequent Elements

JAVASCRIPT Copy

```
function topKFrequent(nums, k) {
  const freq = new Map();
  for (const num of nums) freq.set(num, (freq.get(num) || 0) + 1);

  // Bucket sort by frequency
  const buckets = Array.from({ length: nums.length + 1 }, () ⇒ []);
  for (const [num, count] of freq) buckets[count].push(num);

  const result = [];
  for (let i = buckets.length - 1; i ≥ 0 && result.length < k; i--) {
    result.push( ... buckets[i]);
  }
  return result.slice(0, k);
}
```

Time: O(n). **Space:** O(n).

Frontend connection: "Show the top 10 most viewed products" — frequency counting + top-K extraction.

QUICK CHECK — no marks, just for you.

1. The lesson's Kadane implementation is run on two arrays. What does this print?

```
function maxSubArray(nums) {
  let maxSum = nums[0], currentSum = nums[0];
  for (let i = 1; i < nums.length; i++) {
    currentSum = Math.max(nums[i], currentSum + nums[i]);
    maxSum = Math.max(maxSum, currentSum);
  }
  return maxSum;
}
console.log(maxSubArray([-2, 1, -3, 4, -1, 2, 1, -5, 4]));
console.log(maxSubArray([-3, -1, -7]));
```

- ○ 6
0
- ○ 6
-11
- ○ 6
-1
- ○ 4
-1

Check answer

2. The lesson's two-pass productExceptSelf is run on two arrays, and each result is joined with commas. What does this print?

JAVASCRIPT Copy

```
function productExceptSelf(nums) {
  const n = nums.length;
  const result = Array(n).fill(1);
  let left = 1;
  for (let i = 0; i < n; i++) { result[i] = left; left *= nums[i]; }
  let right = 1;
  for (let i = n - 1; i >= 0; i--) { result[i] *= right; right *= nums[i]; }
  return result;
}
```

- `24,12,8,6`
  `6,0,0`
- `1,2,6,24`
  `1,2,0`
- `24,12,8,6`
  `0,6,0`
- `24,12,8,6`
  `0,0,0`

Check answer

3. According to the lesson, what problem does Reservoir Sampling solve?

- Selecting k items uniformly at random from a stream whose total length is not known in advance.
- Finding the element that appears more times than all other elements combined in a stream.
- Computing the maximum subarray sum of a stream in a single pass using O(1) space.
- Sorting a stream of numbers in linear time by bucketing them on frequency.

Check answer

Frontend System Desi...

LEARN MORE

ARTICLES & DOCS

DSA roadmap (NeetCode)
neetcode.io

VIDEOS



14 patterns to ace coding interviews
YouTube



Ace interviews by thinking in patterns
YouTube



DSA patterns for LeetCode interviews
YouTube

Hand-picked references for this lesson.